

بسم الله الرحمن الرحيم

Sudan University of Science and Technology

Faculty of Graduate Studies



Faculty of Computer Sciences & Information Technology

Enhancing Handwritten Text Recognition Using Transformer Models Derived from Large Language Models

(تعزيز التعرف علي النصوص اليدوية بإستخدام نمازج المحولات المستمدة من نماذج لغوية كبيرة)

A Thesis Submitted in Partial Fulfillment of the Requirements for

M.Sc. degree in Computer Science

By: Elzubier Alsayed Algali

Supervisors:

Dr. Waleed Mergani

November 2026

الآيـــة



صدق الله العظيم

سورة المجادلة – الآية (11)

Declaration

A. Declaration by the student

I hereby declare that this thesis, *Enhancing Handwritten Text Recognition Using Transformer Models Derived from Large Language Models*, is the result of my own research except as cited in the references. The thesis has not been accepted for any degree, or has it been concurrently submitted in candidature for any other degree.

Signature: _____

Name of student: Elzubier Alsayed Algali

Date: _____

B. Declaration by the supervisor

I hereby declare that this thesis, *Enhancing Handwritten Text Recognition Using Transformer Models Derived from Large Language Models*, is the original work of the student, except as cited in the references. To the best of my knowledge, it has not been accepted for any degree, nor concurrently submitted in candidature for any other degree.

Signature: _____

Name of supervisor: Dr. Waleed Mergani

Date: _____

C. Assigning copyright to CGS, SUST (also compulsory in the manual)

I, the undersigned, declare that I am the sole author of this thesis for the Master of Science entitled *Enhancing Handwritten Text Recognition Using Transformer Models Derived from Large Language Models*, which is an original intellectual work. Willingly, I assign the copyright of this work to the College of Graduate Studies (CGS), Sudan University of Science and Technology (SUST). Accordingly, SUST has all the rights to publish this work for scientific purposes.

Candidate's signature: _____

Candidate's name: Elzubier Alsayed Algali

Date: _____

Dedication

To my beloved parents, whose enduring patience, unwavering support, and silent prayers laid the very foundation of this achievement.

To my family, whose constant encouragement and sacrifice sustained me through every stage of this academic journey.

To my esteemed supervisor, Dr. Waleed Mirgani, for his invaluable guidance, scholarly wisdom, and profound mentorship throughout this research.

To the College of Graduate Studies at Sudan University of Science and Technology (SUST), for providing the academic environment and resources that made this milestone possible.

To my teachers and mentors, who not only taught me to read but inspired in me a lifelong devotion to learning.

Acknowledgement

Praise is to Allah, the Almighty, who granted me the health, patience, and knowledge to complete this work.

I would like to thank my supervisor, Dr. Waleed Mergani, for his guidance, encouragement, and careful reading of this thesis. May Allah bless him.

My thanks are also due to the staff of the Faculty of Computer Science and Information Technology, and to the College of Graduate Studies at Sudan University of Science and Technology, for the opportunity to pursue this degree.

I am grateful to my family for their support during the long period of study and writing. I also thank my colleagues and friends who shared advice and time while this research was in progress.

Any remaining shortcomings in this thesis are my own.

Abstract

Handwritten text recognition (HTR) remains challenging because writing styles vary so widely. This thesis evaluates pretrained and fine-tuned TrOCR models, which combine a Vision Transformer (ViT) encoder with a BART text decoder, on the English IAM Handwriting Database benchmark (full test, n=2,915). The study implements a reproducible pipeline measuring Character Error Rate (CER) and Word Error Rate (WER), fine-tunes TrOCR on IAM, trains a CRNN+CTC baseline, evaluates classical Tesseract on the same split, and sets the results against published HTR literature. A Gradio web application demonstrates English line transcription, qualitatively only. The primary pretrained IAM result is CER 4.72% (n=2915). After fine-tuning the CER is 4.96%, slightly higher, and is reported as a negative or neutral finding. The contribution is an aligned research-and-application pipeline rather than a novel architecture.

Keywords: handwritten text recognition; TrOCR; transformers; IAM; CER; fine-tuning; CRNN; Tesseract

المستخلص

لا يزال التعرف على النص المكتوب بخط اليد (HTR) يمثل تحدياً بحثياً بسبب التباين الكبير في أساليب الكتابة. تقيّم هذه الرسالة نماذج TrOCR المدربة مسبقاً والمضبوطة دقيقاً، التي تجمع بين مُرمِّز Vision Transformer (ViT) ومُفكِّك نص BART، على معيار قاعدة بيانات IAM الإنجليزية لسطور الكتابة اليدوية (مجموعة الاختبار الكاملة، n=2,915).

ويُنفَّذ خط أنابيب قابل لإعادة الإنتاج لقياس معدل خطأ الحروف (CER) ومعدل خطأ الكلمات (WER)، ويُضبَط نموذج TrOCR على بيانات IAM، ويُدرَّب خط أساس CRNN+CTC، ويُقيَّم محرك Tesseract الكلاسيكي على نفس التقسيم، وتُوضَع النتائج في سياق الأدبيات المنشورة في مجال HTR. كما يُقدَّم تطبيق ويب باستخدام Gradio لتوضيح نسخ السطور الإنجليزية (بصورة نوعية فقط).

النتيجة الأساسية للنموذج المدرب مسبقاً هي (n 2,915=CER 4.72%)؛ وبعد الضبط الدقيق يبلغ معدل خطأ الحروف 4.96% (أعلى قليلاً؛ ويُبلَّغ عنه بوصفه نتيجة سلبية/محايدة). وتسهم هذه الرسالة بخط أنابيب بحثي–تطبيقي متسق، لا بابتكار معمارية جديدة.

الكلمات المفتاحية: التعرف على الكتابة اليدوية؛ TrOCR؛ المحوّلات؛ قاعدة بيانات IAM؛ معدل خطأ الحروف؛ الضبط الدقيق؛ CRNN؛ Tesseract

Table of Contents

List of figures

List of Tables

List of abbreviations

Chapter One

Introduction

1. CHAPTER I

1.1. Introduction:

Computer vision gives machines a way to interpret images, and character recognition is one of its oldest applications. Handwriting remains among the hardest cases. Letter shapes, slant, spacing, and stroke thickness vary from writer to writer, and the same writer is rarely consistent across a page.

Convolutional neural networks improved image-based recognition substantially and became the standard feature extractor for handwriting systems (Skala et al., 2022). Recognition of full text lines then moved towards recurrent networks trained with Connectionist Temporal Classification, which transcribe a line image without requiring character segmentation (Sahare et al., 2018). That combination defined the state of the art for much of the past decade.

Transformer architectures have since changed the picture. A transformer encoder can model an image as a sequence of patches, and a pretrained text decoder can generate the transcription under a language-model prior. TrOCR is the clearest example of this design: it pairs a Vision Transformer image encoder with a BART text decoder, both initialised from large-scale pretraining. This thesis examines what such a model achieves on a standard English handwriting benchmark, and whether adapting it further on that benchmark is worthwhile.

1.2. Motivation:

Pretrained transformer checkpoints are now published openly and can be applied to handwriting recognition without training a model from scratch. That availability raises a practical question for any researcher with limited compute: how much recognition quality is already available off the shelf, and how much remains to be gained by fine-tuning?

The question matters because reported error rates are difficult to compare across studies. Preprocessing, decoding, and dataset splits differ, and many papers report figures obtained under conditions that are not fully documented. A transparent local protocol, applied to the full benchmark test set rather than a convenient subset, makes it possible to state what a given checkpoint actually delivers.

1.3. Problem Statement:

This research addresses English handwritten text recognition for line images using transformer-based models, evaluated on the IAM Handwriting Database benchmark.

Existing approaches based on convolutional and recurrent networks handle diverse handwriting styles with mixed success, and published comparisons rarely share an identical evaluation protocol. The specific problem is therefore twofold. First, the recognition quality obtainable from a publicly released transformer checkpoint on the full IAM English test partition is not established under a reproducible local protocol. Second, it is unclear whether further fine-tuning on IAM improves that quality, given that the checkpoint is already specialised for English handwriting.

1.4. Aims and Objectives:

1.4.1. Aims:

The aim of this research is to evaluate transformer-based handwritten text recognition on the English IAM Handwriting Database under a transparent and reproducible protocol, and to determine whether fine-tuning the pretrained model on IAM yields a measurable improvement.

The aim is framed as evaluation rather than improvement. A fine-tuning run that fails to reduce the error rate is a valid answer to this question, and is reported as such in Chapters 4 and 5.

1.4.2. Objectives:

1. Review handwritten text recognition methods, with emphasis on transformer architectures and on TrOCR in particular.

2. Implement a reproducible IAM training and evaluation pipeline that measures Character Error Rate and Word Error Rate.

3. Evaluate the pretrained microsoft/trocr-base-handwritten checkpoint and a fine-tuned variant on the full IAM English test partition of 2915 lines.

4. Compare both against a CRNN+CTC baseline trained on the same splits, a classical Tesseract reference evaluated on the same partition, and published IAM figures cited for context only.

5. Deploy a web application that performs English IAM line transcription interactively.

1.5. Research Methodology:

The methodology is stated here in outline. Chapter 3 develops the research design in full, and Chapter 4 reports how it was implemented and what it produced.

1. Collect the data. The study uses the English IAM line corpus distributed as Hugging Face Teklia/IAM-line, which preserves the publisher's official training, validation, and test partitions. Using published splits keeps the results comparable with other work on IAM.

2. Preprocess the images. Line images are converted to RGB and passed to the Hugging Face TrOCR processor, which handles visual normalisation and tokenisation together. No custom binarisation stage is introduced. Mild geometric and photometric augmentation is applied during fine-tuning only, so that the evaluation distribution stays unchanged.

3. Select the model. TrOCR is adopted as the primary architecture, using the microsoft/trocr-base-handwritten checkpoint. Its Vision Transformer encoder reads image patches and its BART decoder generates the transcription. The architecture is used as published; no attention mechanisms, positional encodings, or layers are modified.

4. Apply transfer learning. Fine-tuning starts from the pretrained checkpoint and continues on the IAM training and validation partitions. The objective is teacher-forced cross-entropy. CTC is not used to train TrOCR; it belongs only to the convolutional recurrent baseline.

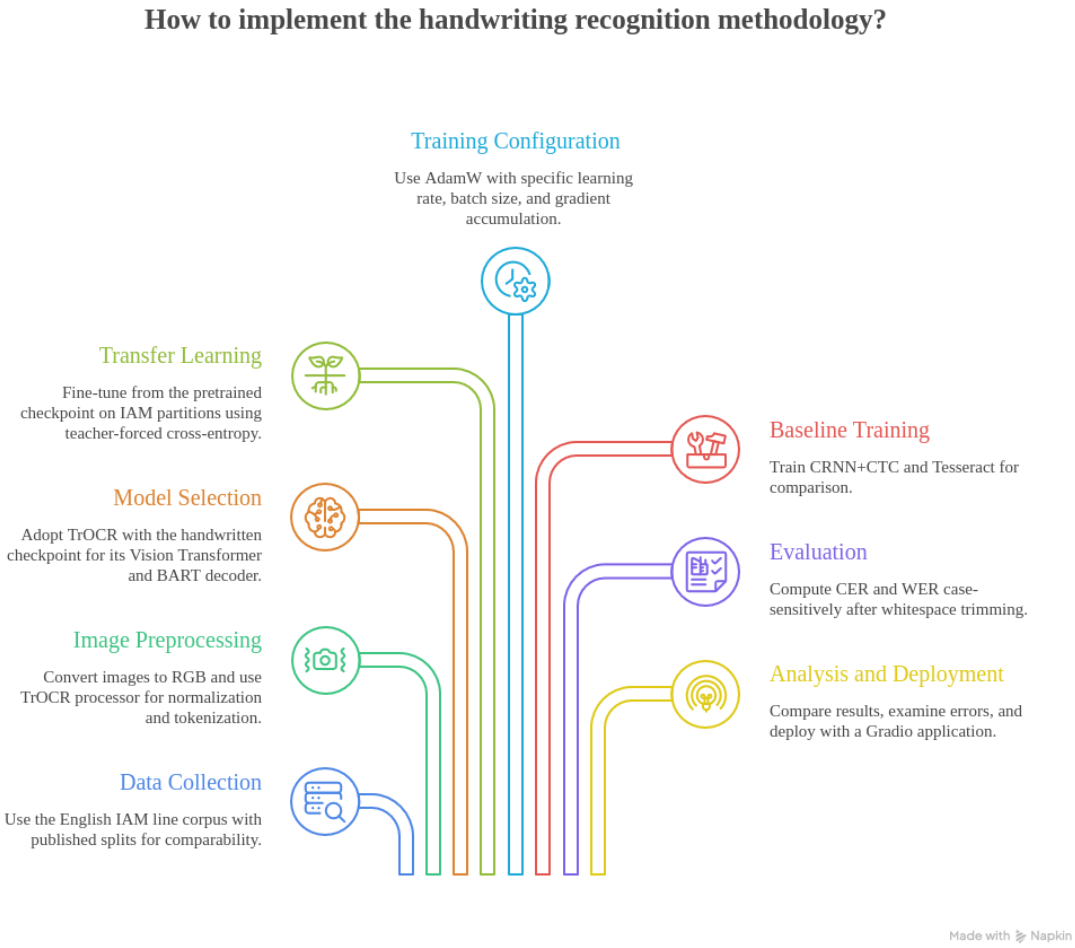
5. Configure training. Optimisation uses AdamW with a learning rate of 5×10^{-6}, a micro-batch of 1, and gradient accumulation of 8 for an effective batch size of 8. Early stopping monitors validation CER. Hyperparameters and wall-clock cost are recorded to experiments/iam_trocr_finetuned/train_log.json.

6. Train the baselines. A CRNN+CTC network is trained from scratch on the same IAM splits as a lightweight neural reference. Tesseract is run in line mode as a classical OCR reference on the same test partition.

7. Evaluate. Character Error Rate is the primary metric and Word Error Rate the secondary one. Both are computed case-sensitively after whitespace trimming, on the complete test partition. Corpus CER is the ratio of total character edits to total reference

characters. Accuracy at the word or character level is not used, because error rate is the established reporting convention in the handwriting recognition literature.

8. Analyse and deploy. Results are compared across the four measured systems, placed against published IAM figures for contextual scale, and examined qualitatively through the highest-error predictions. A Gradio application exposes the recogniser interactively. Demonstration output is qualitative and never contributes a primary error rate.



1.6. Research Scope:

The study covers English handwritten text recognition at the line level, using the IAM Handwriting Database as distributed by Teklia/IAM-line. The corpus provides 6,482 training lines, 976 validation lines, and 2915 test lines. All primary results are measured on the complete test partition.

Four systems are evaluated: pretrained TrOCR, fine-tuned TrOCR, a CRNN+CTC baseline, and Tesseract. A Gradio demonstration accompanies them as an applied component.

The following lie outside the scope. Non-English handwriting is not addressed, nor are historical or degraded document collections. The transformer architecture is not modified. Page-level layout analysis and word segmentation are not performed, since the corpus supplies pre-segmented lines. Published results from other studies are cited for context and are not re-implemented under this protocol.

1.7. Research Planning:

Phase	Work
1	Literature review and research design
2	Dataset acquisition, split preparation, and pipeline implementation
3	Baseline evaluation of the pretrained checkpoint
4	TrOCR fine-tuning on GPU, approximately 90 hours of training
5	CRNN+CTC training and Tesseract evaluation
6	Application development, analysis, and writing

1.8. The Thesis Organization

This research organized as following:

Chapter I: states the problem, the aim, and the objectives, and sets the scope of the study.

Chapter II: reviews handwritten text recognition research, covering convolutional and recurrent approaches and the transformer models that followed them.

Chapter III: presents the research design: the dataset, the preprocessing assumptions, the choice of models, the metric definitions, and the planned experiment matrix. It reports no measured results.

Chapter IV: covers implementation and results. It records the logged hyperparameters, reports the measured error rates on the full test partition, presents the comparison figures, and analyses the outcomes.

Chapter V: concludes the study, summarising the findings, stating the contributions and limitations, and recommending directions for further work.

Appendices describe the principal source modules and provide sample configuration files.

Chapter Two

Literature Review

2. CHAPTER II

2.1. Introduction

Handwritten text recognition has been reshaped by deep learning. Convolutional networks replaced hand-engineered features, recurrent networks trained with Connectionist Temporal Classification made line-level transcription practical, and transformer models have more recently been applied to both the visual and the linguistic side of the task.

This chapter reviews that progression. It covers image processing and optical character recognition as the underlying disciplines, surveys the main handwriting recognition approaches, and examines the studies most relevant to this thesis. Particular attention is given to TrOCR, whose architecture and pretrained checkpoint the present work evaluates.

2.2. Problem Background

Recognising handwritten characters is difficult because the input is highly variable. Size, shape, slant, and spacing differ between writers and within the work of a single writer. Unlike printed text, handwriting offers no fixed glyph inventory to match against.

Convolutional neural networks advanced image-based recognition and became the standard feature extractor for handwriting systems. Recognition then shifted towards recurrent networks combined with Connectionist Temporal Classification, which transcribe an entire line without needing explicit character segmentation. Multidimensional Long Short-Term Memory networks extended this by propagating recurrent connections along both spatial dimensions of the image.

These approaches carry known costs. Recurrent models process sequences step by step, which limits training throughput, and they can require large parameter counts to model long-range dependencies. Transformer architectures address both points through parallel self-attention, and large pretrained language models offer a decoder that already encodes substantial linguistic structure. The question this raises for handwriting recognition, and

the one this thesis takes up, is how much of that pretrained capability transfers to a specific benchmark.

2.3. Images Processing

Image processing covers the operations applied to a digital image to enhance it or to extract information from it. Inputs are images; outputs are either modified images or measurements derived from them (Covington, 2009). The field sits at the intersection of signal processing, engineering, and computer science, and supplies the preparatory operations on which recognition systems depend (Naveenkumar and Ayyasamy, 2016).

For handwriting recognition, the relevant operations are normalisation of size and contrast, noise reduction, and geometric correction. The present study keeps this stage minimal. Line images are converted to RGB and normalised by the TrOCR processor, so that recognition quality reflects the model rather than the image pipeline.

2.4. Optical Character Recognition (OCR)

Optical character recognition converts images of text into machine-encoded characters. An OCR system combines an optical scanner with software that segments and classifies the resulting image regions. Once converted, the text can be searched, edited, stored compactly, or passed to downstream processing such as translation or text mining.

Early systems were calibrated to a single font and required a template for each character. Modern engines handle a wide range of fonts and can reproduce page structure including columns and embedded images. Recognition of printed text is largely a solved problem in well-conditioned settings.

Handwriting remains the harder case. General-purpose OCR engines are trained predominantly on printed glyphs and degrade sharply on unconstrained handwriting, which is why handwriting recognition has developed as a distinct research area with its own architectures and benchmarks (Isheawy and Hasan, 2015). This thesis measures that gap directly by evaluating Tesseract on the same test partition as the neural systems.

2.5. Handwritten Text Recognition.

Handwritten text recognition transcribes images of handwriting into machine-readable text. It is commonly divided by input granularity into character, word, and line recognition, and by acquisition mode into online recognition, which records pen trajectory, and offline recognition, which works from a static image. This thesis addresses offline line recognition.

The dominant offline approach for much of the past decade combined convolutional feature extraction with recurrent sequence modelling and a CTC output layer (Sahare et al., 2018). CTC removes the need for character-level alignment, allowing a model to be trained on line images paired only with their transcriptions. Multidimensional LSTM variants extended this to capture context in two dimensions (Skala et al., 2022).

Applications include the digitisation of archives and the preservation of historical documents, where manual transcription is prohibitively slow. Recognition quality is reported almost universally as Character Error Rate and Word Error Rate, computed as normalised edit distances. These are the metrics adopted in this thesis.

2.6. Related Handwritten Text Recognition (HTR) Researches

Historical document transcription. Parres and Paredes (2023) fine-tune vision encoder-decoder transformers for historical manuscript transcription, reporting Word Error Rates of 6.9% on ICFHR 2014 Bentham, 14.5% on ICFHR 2016 Ratsprotokolle, and 17.3% on Saint Gall. Their work shows that transformer fine-tuning transfers to degraded historical material, though the corpora and language differ from the English IAM benchmark used here.

TrOCR. Li et al. (2023) propose TrOCR, an end-to-end transformer OCR model that pairs a pretrained Vision Transformer image encoder with a pretrained BART text decoder. Because the decoder is a language model, no separate post-processing language model is required. The authors report that pretrained vision and language components improve recognition, and that the pure transformer design outperforms both CRNN systems and Tesseract on their benchmarks. TrOCR is the architecture evaluated in this thesis, and the microsoft/trocr-base-handwritten checkpoint is its published English handwriting variant.

Handwriting generation. Bhunia et al. (2021) introduce Handwriting Transformers, a transformer network for styled handwritten text image generation that models both global and local style patterns. The work addresses synthesis rather than recognition, but demonstrates that transformer attention captures handwriting style effectively.

Attention-based recognition. Kass and Vats (2022) present AttentionHTR, an attention-based sequence-to-sequence model for handwritten word recognition that uses ResNet feature extraction with bidirectional LSTM sequence modelling. Transfer learning from scene text images mitigates the scarcity of handwriting training data. On IAM they report a CER of 6.50% and a WER of 15.40% at word level.

Deep learning on IAM. Balci et al. (2017) and Manchala et al. (2020) both apply convolutional and recurrent architectures, including VGG-19 and ResNet-18/34, to word-level classification and character segmentation on IAM. Both report word-level classification accuracy above 90.3%. That figure is an accuracy measure under each study's own segmentation protocol and is not directly comparable with the corpus-level error rates reported in this thesis.

Lightweight transformers. Barrere et al. (2022) propose a Light Transformer architecture that combines convolutional layers with transformer components while keeping the parameter count low enough for small datasets. A hybrid loss combining CTC and cross-entropy improves training efficiency. The model reaches a CER of 5.70% on the IAM test set without additional training data, improving to 4.76% when synthetic data is added.

Gated convolutional models. de Sousa Neto et al. (2020) introduce HTR-Flor, a gated CNN with bidirectional GRUs that outperforms comparable CRNN systems on IAM with fewer parameters. Coquenet et al. (2020) go further and remove recurrence entirely: their Gated Fully Convolutional Network uses depthwise separable convolutions and a gating mechanism, reaching a CER of 4.35% on RIMES and 7.99% on IAM with a low parameter count.

Position of this thesis. These studies establish that both convolutional and transformer designs achieve single-digit CER on IAM, and that pretraining is the main lever available to a study without large-scale compute. What they do not settle is how a published transformer checkpoint performs on the full IAM test partition under a protocol that is

documented end to end, or whether further fine-tuning on that same benchmark adds anything. That is the gap this thesis addresses.

2.7. Related HTR Researches Summary Table:

#	Study	Objective	Dataset	Tools and techniques	Reported results	Limitation
1	Fine-Tuning Vision Encoder-Decoder Transformers for HTR on Historical Documents (Parres and Paredes, 2023)	Assess fine-tuned vision encoder-decoder transformers for historical manuscript transcription	ICFHR 2014 Bentham; ICFHR 2016 Ratsprotokolle; Saint Gall	Fine-tuning of pretrained vision encoder-decoder transformers	WER 6.9% (Bentham), 14.5% (Ratsprotokolle), 17.3% (Saint Gall)	No comparison against other fine-tuning strategies for vision transformers
2	TrOCR: Transformer-Based Optical Character Recognition with Pre-trained Models (Li et al., 2023)	Propose an end-to-end transformer OCR model built from pretrained vision and language components	Synthetic handwritten text-line images generated with TRDG from 5,427 handwriting fonts; evaluated on IAM and further printed, handwritten, and scene-text benchmarks	ViT image encoder + BART text decoder; pretrained vision and language components	Pretrained components improve recognition; the transformer design outperforms CRNN systems and Tesseract	Limited comparison against other transformer-based OCR models
3	Handwriting Transformers	Generate styled handwritten text	Not applicable (generation task)	Transformer encoder-decoder	Produces realistic styled handwritt	Limited comparison with existing

	(Bhunia et al., 2021)	images capturing global and local style		with attention (HWT)	en text images	styled text generation methods
4	Attention HTR (Kass and Vats, 2022)	Attention-based sequence-to-sequence handwritten word recognition using transfer learning from scene text	IAM; Imgur5K	ResNet feature extraction with bidirectional LSTM sequence modelling and content-based attention	IAM CER 6.50%; IAM WER 15.40%	Does not examine the effect of different pretrained models
5	Handwritten Text Recognition using Deep Learning (Balci et al., 2017)	Classify handwritten words using CNNs and LSTMs for document conversion	IAM Handwriting Dataset, 1,500+ forms, 600+ writers	VGG-19 and ResNet-18/34 with padding and rotation augmentation; Adam optimiser	Word-level accuracy above 90.3%; character-level classification aids word recognition	Does not explore transformer architectures
6	Handwritten Text Recognition using Deep Learning with TensorFlow (Manchala et al., 2020)	Build an HTR system for word classification in digital conversion	IAM Handwriting Dataset, 1,500+ forms, 600+ writers	VGG-19 and ResNet-18/34 for word classification; LSTMs for character segmentation; Tesseract for preprocessing	Accuracy above 90.3%	Limited comparison against other HTR systems
7	A Light Transformer-Based Architecture for	Achieve competitive HTR on small datasets	IAM Handwriting Dataset,	CNN and transformer hybrid; combined CTC and	CER 5.70% on the IAM test set without	Provides no qualitative analysis of

	HTR (Barrere et al., 2022)	with a lightweight transformer	text-line images	cross-entropy loss	additional data; CER 4.76% with additional synthetic data	generated outputs
8	HTR-Flor (de Sousa Neto et al., 2020)	Optimise offline HTR with a gated CRNN using fewer parameters	IAM Handwriting Database	Gated CNN with bidirectional GRUs, implemented in PyTorch and TensorFlow	Outperforms comparable CRNN systems with a reduced parameter count	Limited explanation of the gated architecture design choices
9	Recurrence-free unconstrained HTR using a gated fully convolutional network (Coquenet et al., 2020)	Replace LSTM-based modelling with a gated fully convolutional network	RIMES; IAM, images resized to 64 pixels in height	GFCN with CTC loss, depthwise separable convolutions, Adam optimiser, CER and WER evaluation	CER 4.35% on RIMES; CER 7.99% on IAM	No detailed analysis of the learned features and filters

2.8. Summary

This chapter traced handwritten text recognition from convolutional feature extraction, through recurrent models trained with CTC, to transformer architectures built on pretrained components. Image processing and optical character recognition were reviewed as the underlying disciplines, and the persistent difficulty of unconstrained handwriting for general-purpose OCR engines was noted.

The reviewed studies establish two points that shape the present work. Single-digit CER on IAM is achievable by several architectural families, with reported figures including 5.70% for a light transformer, 6.50% for an attention-based model, and 7.99% for a gated fully convolutional network. Pretraining is the principal lever available to a study without large-scale training resources, and TrOCR is the clearest instance of a model built entirely from pretrained vision and language components.

What the literature leaves open is the performance of a published TrOCR checkpoint on the complete IAM English test partition under a fully documented local protocol, and whether fine-tuning on that benchmark improves it. Chapter 3 sets out the research design that addresses those questions.

Chapter Three

Methodology

3. CHAPTER III

3.1. Introduction

A sound empirical study of handwritten text recognition (HTR) requires more than the selection of a fashionable model: it requires an explicit account of the data, the preprocessing assumptions, the learning objectives, the evaluation criteria, and the comparisons against which claims will be judged. This chapter therefore sets out the methodological framework adopted for the present thesis. The research focuses on English line-level recognition using the IAM Handwriting Database and investigates transfer learning from a pretrained TrOCR transformer, whose Vision Transformer (ViT) encoder is paired with a BART text decoder. A convolutional recurrent network trained with Connectionist Temporal Classification (CRNN+CTC) provides a neural baseline of classical design, while Tesseract is retained as an optional classical optical character recognition reference when the runtime environment permits its use.

The chapter proceeds from dataset characterisation to preprocessing, model choice, metric definitions, and the planned experiment matrix. In keeping with conventional thesis organisation, no experimental scores are reported here; Chapter 4 is responsible for implementation detail, measured outcomes, and interpretive analysis.

3.2. Dataset

Reliable benchmarking in HTR depends on a corpus whose splits are stable and publicly documented. The study therefore employs the English IAM line corpus distributed as Hugging Face Teklia/IAM-line, which preserves the publisher's official training, validation, and test partitions. Each sample consists of a line image together with its corresponding transcription. Local preparation yields organized image directories and label tables suitable for both training and evaluation, while a small demonstration subset is reserved for the interactive application described later in the thesis.

Property	**Value**
Language	English
Unit	Line images
Train / Val / Test	6,482 / 976 / 2,915
Modalities	PNG line image + transcription text
Source	Hugging Face Teklia/IAM-line

Table 3.1 — IAM line dataset (Teklia/IAM-line)

The test partition contains 2,915 lines. This full split constitutes the primary evaluation population of the thesis and is preferred over reduced samples that would weaken comparability with published IAM practice.



Figure 3.1 — Example IAM line

A representative English line image illustrating the visual domain addressed by the study.

3.3. Preprocessing

Preprocessing is kept intentionally conservative so that recognition performance can be attributed primarily to the model rather than to aggressive image engineering. Line images are converted to RGB and presented to the Hugging

Face TrOCR processor, which jointly handles visual normalisation and tokenisation for the encoder–decoder model. No custom binarisation pipeline is introduced. Mild geometric and photometric augmentation is applied exclusively during fine-tuning, on the premise that stochastic variation may aid adaptation while leaving the evaluation distribution unaltered. All reported test scores are therefore computed on unaugmented images.

3.4. Model selection

3.4.1. TrOCR as the primary model

TrOCR is adopted as the principal recognition architecture because it exemplifies the contemporary transfer-learning paradigm in HTR: an image transformer encodes local visual structure, while an autoregressive text transformer decodes character sequences under a language-model prior. The study begins from the publicly available checkpoint microsoft/trocr-base-handwritten, which has already been pretrained for English handwritten lines. Fine-tuning on the IAM training and validation partitions uses teacher-forced cross-entropy rather than CTC, with AdamW optimisation and early stopping guided by validation CER. This design choice is consequential: CTC remains associated only with the neural baseline, and is never treated as the training objective of TrOCR.

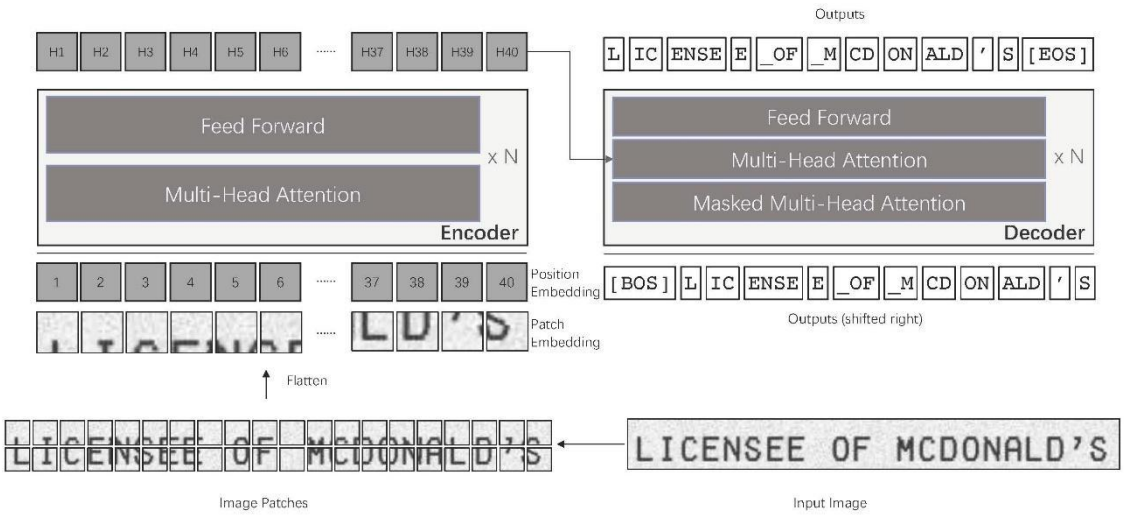


Figure 3.2 — TrOCR architecture (conceptual)

Image patches are encoded by a ViT encoder and decoded by a cross-attentive BART decoder into a token sequence.

3.4.2. CRNN+CTC as a neural baseline

To situate transformer performance against a familiar neural alternative, a CRNN with CTC is trained from scratch on the same IAM splits. The baseline is deliberately lightweight: convolutional feature extraction is followed by bidirectional recurrent modelling and CTC alignment. Its purpose is comparative clarity rather than architectural rivalry. A large performance gap, should it appear, would illuminate the benefit of large-scale pretrained transformers under matched data conditions.

3.4.3. Tesseract as an optional classical baseline

Tesseract, operated in line-oriented recognition mode, is included as a classical OCR reference. Because availability of the system binary can vary across machines, the methodology treats Tesseract as optional: when execution is impossible, Chapter 4 records a skipped status instead of fabricating scores. This rule preserves methodological honesty without forcing an incomplete software dependency to dictate the scientific narrative.

3.5. Experiment design and evaluation metrics

The empirical programme is organised as a small matrix of complementary runs. Primary scientific claims rest on full-test evaluation of pretrained TrOCR, fine-tuned TrOCR, and the CRNN+CTC baseline. The optional Tesseract run and the demonstration subset support completeness and application discussion, respectively, but do not redefine the primary evidence base.

ID	Run	Role	Evaluation sample
E1	Pretrained TrOCR	Primary transformer baseline	Full test (n=2,915)
E2	Fine-tuned TrOCR	Adaptation experiment	Full test (n=2,915)
E3	CRNN+CTC	Neural baseline	Full test (n=2,915)
E4	Tesseract	Optional classical baseline	Full test (n=2,915)
E5	Demonstration subset	Application support	Demo lines only

Table 3.2 — Designed experiment matrix

Two metrics dominate the evaluation. Character Error Rate (CER) quantifies the normalised edit distance at character level and remains the principal indicator of

transcription fidelity in HTR. Word Error Rate (WER) provides a complementary word-level view of recognition difficulty. Both are computed after case-sensitive whitespace normalisation; transcriptions are not lowercased, so that the metric reflects orthographic detail rather than an artificially simplified string space. Corpus CER is defined as the ratio of total character edits to total reference characters across the evaluated set.

Engineering smoke tests with severely reduced sample limits may be used during development, but they are excluded from thesis claims. All numbers advanced as research evidence must originate from the full GPU protocol on the complete test partition.

3.6. Conclusion

This chapter has established the methodological foundation of the thesis. The research design centres on English line-level recognition with the IAM Handwriting Database, using the official training, validation, and test partitions so that evaluation rests on a complete and publicly documented population. Preprocessing remains deliberately light, preserving the primacy of the recognition model rather than of custom image engineering. TrOCR is adopted as the principal transfer-learning system, while CRNN+CTC supplies a controlled neural baseline and Tesseract is retained only as an optional classical reference. Character and word error rates, computed under a shared case-sensitive definition, constitute the criteria by which later claims will be judge

Chapter Four

Implementation & Results

4. CHAPTER IIII

4.1. Purpose and contribution of the chapter

Having defined the research design in Chapter 3, I now report how I realised that design and what I observed. I frame my contribution as an aligned research-and-application study rather than as the invention of a new HTR architecture. Concretely, I implemented a reproducible English IAM training and evaluation workflow; I fine-tuned TrOCR on the official training and validation partitions and logged the run in full; I compared the transformer against a CRNN+CTC neural baseline and classical Tesseract on the identical full test set of 2,915 lines; and I built an interactive demonstration that places the recogniser in an applied setting. I also cite published IAM CER figures for contextual scale, without claiming protocol-identical superiority.

4.2. Implementation overview

I organised the implementation around the methodological commitments of Chapter 3. I load line images and transcriptions for training and evaluation; I instantiate either the public TrOCR checkpoint or my locally fine-tuned directory for generation; I adapt TrOCR with cross-entropy, AdamW, and early stopping on validation CER; I train and evaluate a separate CRNN+CTC baseline; I compute case-sensitive CER and WER under a shared metric definition; I execute the experiment matrix through evaluation runners; and I expose recognition through a Gradio interface for interactive demonstration. I place detailed module listings in the appendix so that this chapter can remain focused on scientific substance rather than software documentation.

Component	Role in the study
Dataset I/O	Provides line images and transcriptions for train/eval
TrOCR loading and generation	Instantiates Hub or local checkpoints for inference
TrOCR fine-tuning	Adapts the pretrained model on IAM train/val
CRNN+CTC baseline	Trains and evaluates the neural reference system
Metrics	Computes CER and WER under a shared definition
Evaluation runners	Executes the designed experiment matrix
Demonstration interface	Supports interactive English line transcription

Table 4.1 — Principal implementation components

Technologies covered in detail

Technology	Key reason documented
Python 3.9+/3.10	ML ecosystem standard; required by all dependencies
PyTorch + torchvision	TrOCR/Hugging Face native runtime; CUDA support
Hugging Face Transformers	Official TrOCR implementation; avoids reimplementing ViT+BART
Hugging Face Hub	Model distribution without storing weights in Git
TrOCR (ViT + BART)	Thesis focus model; IAM-pretrained checkpoint
Gradio 4.x	ML demo UI without custom frontend development
Pillow	Standard image format for Gradio + TrOCRProcessor
jiwer	Dedicated CER/WER library; chosen over manual or evaluate
Hugging Face datasets	Official IAM splits from Teklia/IAM-line
Parquet + PyArrow	Efficient local caching between download and export
pandas	Tabular analysis of experiment outputs
PyYAML	Reproducible, human-readable experiment configs
Tesseract + pytesseract	Classical baseline on identical IAM data
matplotlib	Thesis CER comparison figures

setuptools / pyproject.toml	Editable install; explicit dependencies
JSON + CSV	Traceable, tool-agnostic experiment artifacts
functools.lru_cache	Avoid reloading model on every Gradio click

Table 4.2 summarizes the technology stack.

4.3. Experimental procedure

4.3.1. TrOCR fine-tuning

Fine-tuning began from microsoft/trocr-base-handwritten and used the full IAM training and validation sets (6,482 and 976 lines). Optimisation employed AdamW with a learning rate of 5×10^{-6}, a micro-batch size of 1, and gradient accumulation of 8, yielding an effective batch size of 8. A maximum of eight epochs was planned, with early-stopping patience of two validations. Training ran on CUDA and required approximately 88.7 hours of wall-clock time.

Under this schedule, validation CER reached its best value of 0.0137 after the first epoch and then deteriorated (0.0162 in epoch 2; 0.0187 in epoch 3). Early stopping therefore terminated training after three epochs, and the exported model corresponds to the best validation checkpoint rather than the final epoch weights. This behaviour is material to later interpretation: the adaptation run was completed and logged, yet it did not exhibit sustained validation improvement.

4.3.2. CRNN+CTC training

The CRNN baseline was trained from scratch on the same IAM partitions with learning rate 1×10^{-3}, batch size 8, and fixed input geometry of 32×128. Eight epochs were executed under early-stopping patience, with a character set of size 79. Best validation CER remained high (approximately 0.80), already signalling limited competitiveness relative to pretrained TrOCR before test evaluation.

4.3.3. Evaluation matrix

Evaluation followed the design of Chapter 3. Measured full-test claims rest on pretrained TrOCR (E1), fine-tuned TrOCR (E2), CRNN+CTC (E3), and Tesseract (E4), each assessed on the identical IAM test split of 2,915 lines. The demonstration run (E5) supports the application section only: it is qualitative, may use a tiny demo subset, and is never used as a primary CER source for the thesis.

4.4. Results

Table 4.2 reports the primary measured outcomes.

Method	**CER**	**WER**	**n**
TrOCR pretrained	4.72%	11.64%	2915
TrOCR fine-tuned	4.96%	12.46%	2915
CRNN+CTC	81.82%	96.56%	2915
Tesseract	56.33%	91.35%	2915

Table 4.3 — Measured results on the full IAM English test set

The pretrained transformer attains a CER of 4.72% and a WER of 11.64%. After fine-tuning, CER and WER become 4.96% and 12.46%, respectively—slightly worse than the Hub checkpoint under the logged early-stopped schedule. The lightweight CRNN+CTC baseline yields a CER of 81.82% and a WER of 96.56%. Classical Tesseract, evaluated in line mode on the same split, yields a CER of 56.33% and a WER of 91.35%.

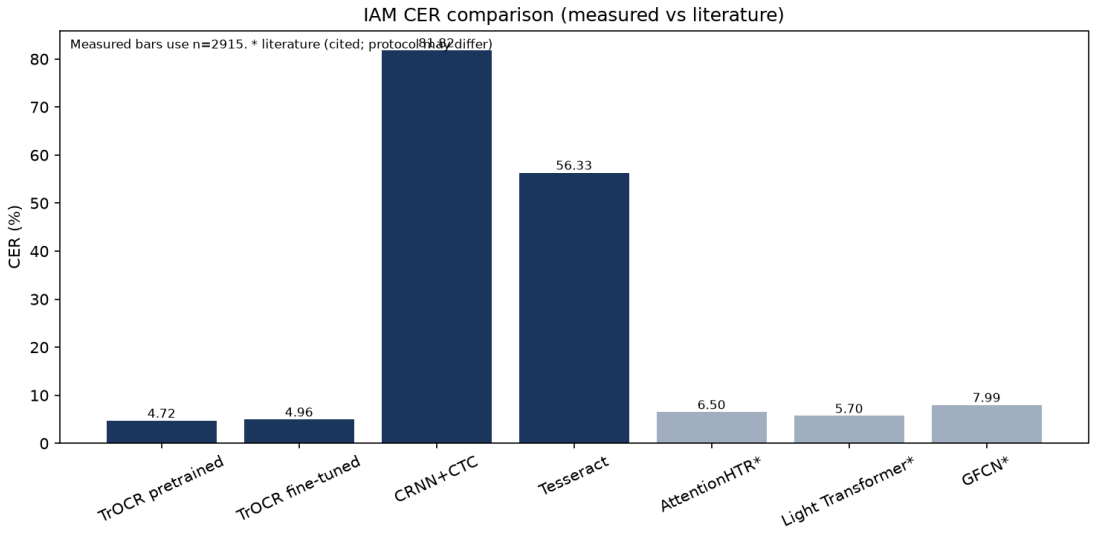


Figure 4.1 — CER comparison

Measured CER values for the systems evaluated in this thesis, with selected literature figures shown separately for contextual comparison.

Method	CER (IAM)	Source
AttentionHTR	6.50%	Kass & Vats, 2022
Light Transformer	5.70%	Barrère et al., 2022
GFCN	7.99%	Coquenet et al., 2020

Table 4.4 — Literature CER figures cited for context

These published values help the reader locate the present scores within the broader IAM literature. They are not re-implemented under the preprocessing and split conventions of this repository, and the thesis therefore does not assert a matched state-of-the-art ranking.

4.5. Discussion and analysis

4.5.1. Pretrained TrOCR as the principal empirical result

The pretrained TrOCR evaluation constitutes the central positive finding of the study. A CER of 4.72% on 2,915 previously unseen IAM lines indicates that a publicly released transformer HTR checkpoint can deliver strong line-level recognition when assessed under a complete and transparent local protocol. For a master's investigation concerned

with transfer learning rather than architectural invention, this result is scientifically meaningful: it shows what current pretrained models already achieve before any student-led adaptation is attempted.

4.5.2. Interpretation of the fine-tuning outcome

Fine-tuning did not improve test recognition. The adapted model's CER of 4.96% is slightly higher than that of the Hub checkpoint. Several considerations make this outcome intelligible. First, the starting checkpoint is already specialised for English handwriting, so residual headroom on IAM is limited. Second, validation CER worsened after the first epoch, and early stopping rightly preferred the earlier checkpoint; continued training under the logged schedule was not justified by the validation signal. Third, the optimisation used a small micro-batch with gradient accumulation, which may be adequate for a feasibility study yet suboptimal for extracting further gains. The appropriate academic reading is therefore that fine-tuning was executed and measured, and that its effect under these conditions was neutral to slightly adverse—not that adaptation is universally unhelpful, nor that the experiment failed to run.

4.5.3. The CRNN+CTC baseline in perspective

The CRNN+CTC system's CER of 81.82% confirms a substantial gap relative to TrOCR under matched data and metrics. This outcome should be read carefully: the baseline is intentionally lightweight—a compact CNN–BiLSTM–CTC model trained from scratch with fixed 32×128 geometry—and is not presented as an optimised competitive neural HTR system. Its role is comparative anchoring. The large error rate therefore illustrates the advantage of large-scale pretrained transformers in this setting; it does not imply that every CTC architecture would fail equally.

4.5.4. Classical Tesseract as a reference baseline

Tesseract provides a classical OCR point of comparison on the same full test partition, reaching CER 56.33% and WER 91.35%. The gap to pretrained TrOCR is large, as expected when a general-purpose OCR engine is applied to unconstrained handwriting line images without handwriting-specific adaptation. The result strengthens the case for

transformer HTR while remaining modest in claim: Tesseract is a classical reference, not a tuned handwriting specialist under this protocol.

4.5.5. Error behaviour and qualitative observations

Inspection of high-CER predictions reveals recurring difficulties with very short references, punctuation-dominated or dash-like strings, and rare or hyphenated tokens. Such lines can inflate per-example CER even when corpus-level error remains comparatively low. These observations caution against over-interpreting isolated failures and reinforce the primacy of aggregate CER/WER on the full test partition.

4.5.6. Limitations affecting interpretation

Interpretation remains bounded by the study's design. The work addresses English line-level IAM only. Literature comparisons are contextual rather than protocol-matched. The fine-tuning schedule was short and early-stopped, and should not be mistaken for an exhaustive hyperparameter search. The CRNN baseline was kept simple by design. Application/demo scores—especially any smoke or synthetic subset metrics—are illustrative only and are excluded from Table 4.2 and from primary thesis claims.

4.6. Application prototype

Beyond offline evaluation, the study includes a Gradio-based demonstration for interactive English line transcription, optionally with reference-based CER when ground truth is supplied. The prototype is an applied complement to the experimental programme: it shows that the recognition stack can be used by a non-specialist interface. Scientific claims remain dependent on the full-test E1–E4 runs reported above, not on demo-subset CER.

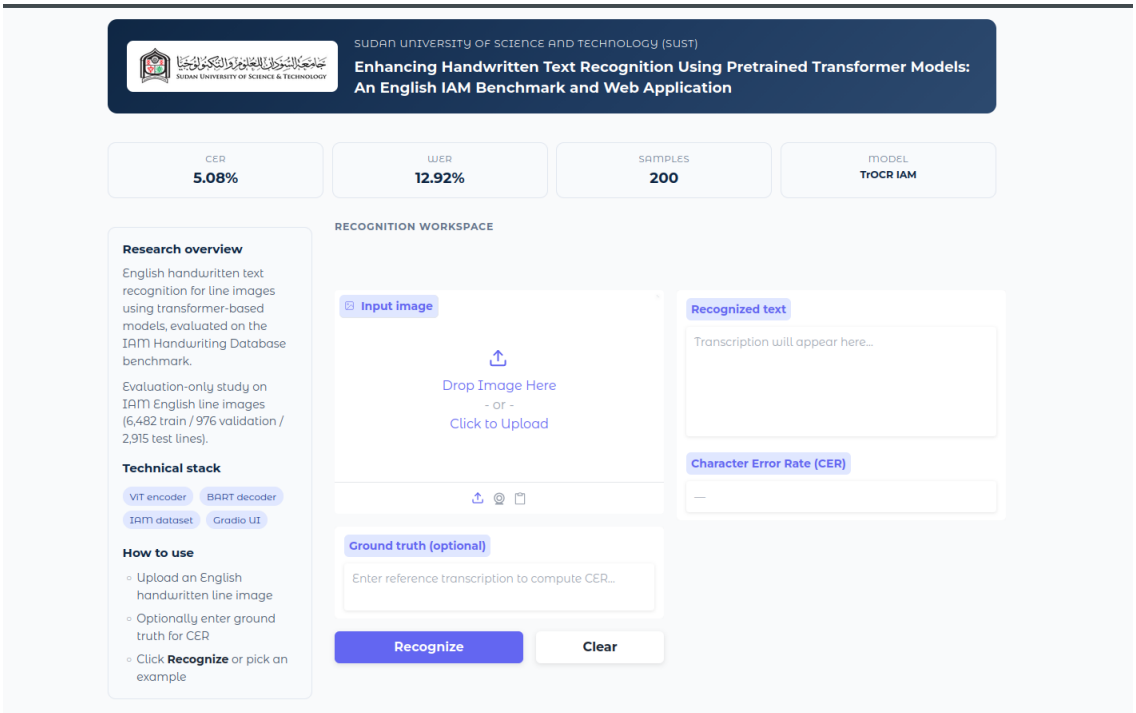


Figure 4.2 — Application Gradio UI

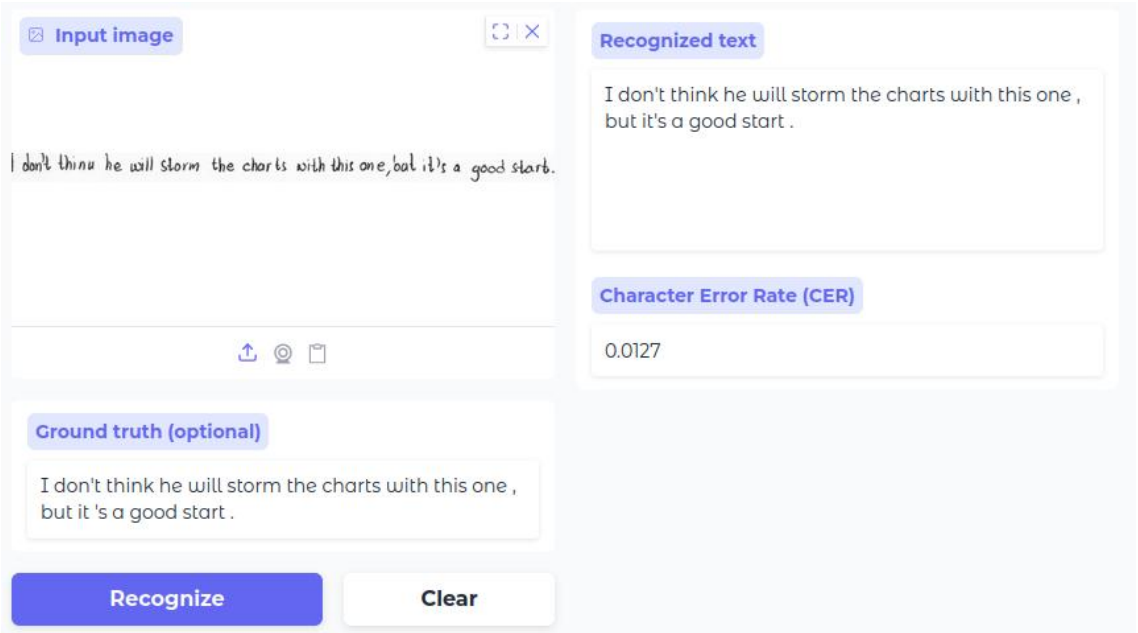


Figure 4.3 — Application Gradio UI - example

Chapter Five

Conclusion & Future Work

5.CHAPTER V

5.1. Revisiting the research aim

This thesis set out to establish how well a transformer-based handwritten text recognition (HTR) model performs on a standard English benchmark when it is evaluated under a transparent and locally reproducible protocol. The central question was practical as much as scientific. Given a widely used pretrained TrOCR checkpoint, pairing a Vision Transformer (ViT) image encoder with a BART text decoder, what recognition quality is obtainable on the IAM Handwriting Database, and does further fine-tuning on the official IAM splits produce a measurable gain? Two subsidiary questions followed. How does such a model compare with a conventional neural sequence baseline (CRNN with CTC) and with a classical OCR engine (Tesseract) under identical test conditions? And how can the resulting system be presented as an interactive application?

The investigation was scoped narrowly. It covers English line-level recognition on IAM, with Character Error Rate (CER) and Word Error Rate (WER) as the primary metrics. No new network architecture was proposed. The goal was instead a single aligned workflow running from data preparation through training and evaluation to a working demonstration, with each stage logged and the outcomes reported with academic care.

5.2. Principal findings

The experiments were conducted on the full IAM English test partition of 2,915 line images. Table 5.1 summarises the main quantitative outcomes. Demo-subset and smoke metrics are excluded from it.

Table 5.1 — Measured outcomes on the full IAM English test set

Method	**CER**	**WER**	**n**
TrOCR pretrained	4.72%	11.64%	2915
TrOCR fine-tuned	4.96%	12.46%	2915
CRNN+CTC	81.82%	96.56%	2915
Tesseract	56.33%	91.35%	2915

The strongest result came from the pretrained TrOCR model, which reached a CER of 4.72% and a WER of 11.64%. A publicly available transformer HTR checkpoint can

therefore deliver competitive line-level recognition on IAM, provided it is assessed on the complete test split rather than a reduced sample.

Fine-tuning on the IAM training and validation sets completed successfully and the run was documented in full. Under the logged schedule, however, the adapted model did not surpass the Hub checkpoint on the held-out test set. Its CER rose slightly to 4.96%, with a WER of 12.46%. Early stopping halted training after three epochs once validation CER ceased to improve. The outcome is informative rather than disappointing. The base checkpoint is already specialised for English handwriting, so the remaining margin is narrow, and a short adaptation run with a small micro-batch may in any case be insufficient to unlock further gains. Fine-tuning is best read as a completed experimental intervention whose measured effect was neutral to slightly adverse, not as an automatic route to higher accuracy.

The CRNN+CTC baseline, trained from scratch on the same splits, produced a markedly higher CER of 81.82%. That figure reflects a lightweight CNN, BiLSTM and CTC reference with constrained input geometry. It is not a claim about the best achievable CTC system. The contrast does nevertheless underline the advantage of large-scale pretrained transformers under matched data and metrics.

Classical Tesseract, evaluated in line mode on the identical full test partition, yielded a CER of 56.33% and a WER of 91.35%. As a general-purpose OCR engine without handwriting-specific adaptation, it remained far behind TrOCR, completing the baseline triad set out in the methodology.

Published IAM figures from related studies were cited for contextual scale only. Those works may differ in preprocessing, decoding, or split definitions, so no matched state-of-the-art ranking is asserted here. Inspection of the high-error lines showed that short references, punctuation-heavy strings, and rare or hyphenated tokens remain difficult even when corpus-level CER is low. The Gradio demonstration showed that the same recognition stack can be exposed for interactive use. Its demo-subset scores count as application evidence only and do not enter the primary result tables.

5.3. Contributions of the study

Within the limits of a master's project, the contributions are as follows. The first is a coherent English IAM evaluation protocol, under which pretrained TrOCR, fine-tuned TrOCR, CRNN+CTC, and Tesseract are all assessed on the identical full test partition with shared metric definitions. The second is a fully documented end-to-end fine-tuning experiment, covering learning rate, batching strategy, early stopping, and wall-clock cost, so that the adaptation step is reproducible and open to scrutiny. The third is the reporting of a negative or near-neutral fine-tuning outcome with the same clarity as a positive result, which strengthens the credibility of the empirical account. The fourth is the positioning of transformer performance against both a lightweight neural baseline and a classical OCR baseline, without overstating either comparator. The fifth is a lightweight web demonstration that places the recognition model in an applied setting while keeping demo behaviour separate from full-test evidence. Taken together these amount to an aligned research-and-application contribution rather than a claim of architectural novelty.

5.4. Limitations of the study

Several constraints qualify the findings. The work addresses English line images only; page-level layout analysis and multilingual handwriting fall outside its remit. The fine-tuning regime was short and early-stopped, and cannot be presented as an exhaustive search over optimisation choices. The CRNN baseline was kept simple by design. More elaborate neural sequence models might narrow the gap to TrOCR, but pursuing them would have drawn the study away from its transformer-centred question. Tesseract was run as a classical line-mode reference rather than as a handwriting-tuned specialist. Literature comparisons remain secondary and protocol-sensitive. Application metrics gathered on the small demo subset, including any smoke or synthetic checks, are illustrative and were never intended to replace full-test evaluation.

5.5. Recommendations for future research

TFurther work could pursue longer or more carefully regularised fine-tuning schedules, testing alternative learning-rate policies and larger effective batch sizes to determine whether the observed plateau can be overcome. Stronger neural baselines trained under the same IAM splits would sharpen the comparative picture beyond the lightweight

CRNN used here. Beyond the line level, the pipeline could be extended towards page-level HTR, and the interface towards batch processing and structured error review, increasing both scientific reach and practical utility.

5.6. Concluding remarks

This study has shown that pretrained transformer-based HTR can be evaluated rigorously on the full English IAM test set within a transparent student research pipeline, attaining a CER of 4.72%. Additional fine-tuning was executed successfully, yet under the logged conditions it did not improve test recognition, and that finding is reported as measured. Neural and classical baselines place the result in comparative context, while the demonstration interface completes the applied thread of the project. The lasting value of the work lies in that measured alignment of method, experiment, and application, not in a claim of architectural novelty.

References

References

- Balci, B., Saadati, D. and Shiferaw, D. (2017) 'Handwritten text recognition using deep learning', CS231n: Convolutional Neural Networks for Visual Recognition, Stanford University, Course Project Report.
- Barrere, K., Soullard, Y., Lemaitre, A. and Coüasnon, B. (2022) 'A light transformer-based architecture for handwritten text recognition', in Document Analysis Systems (DAS 2022). Lecture Notes in Computer Science, vol. 13237. Cham: Springer, pp. 275–290. Available at: https://doi.org/10.1007/978-3-031-06555-2_19
- Bhunia, A.K., Khan, S., Cholakkal, H., Anwer, R.M., Khan, F.S. and Shah, M. (2021) 'Handwriting transformers', in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). IEEE, pp. 1376–1385.
- Coquenet, D., Chatelain, C. and Paquet, T. (2020) 'Recurrence-free unconstrained handwritten text recognition using gated fully convolutional network', in 17th International Conference on Frontiers in Handwriting Recognition (ICFHR). IEEE, pp. 19–24. Available at: https://doi.org/10.1109/ICFHR2020.2020.00015
- Covington, M.A. (2009) Digital SLR Astrophotography. Cambridge: Cambridge University Press.
- de Sousa Neto, A.F., Bezerra, B.L.D., Toselli, A.H. and Lima, E. (2020) 'HTR-Flor: a deep learning system for offline handwritten text recognition', in 33rd SIBGRAPI Conference on Graphics, Patterns and Images (SIBGRAPI). IEEE, pp. 54–61. Available at: https://doi.org/10.1109/SIBGRAPI51738.2020.00016
- Isheawy, N.A.M. and Hasan, H. (2015) 'Optical character recognition (OCR) system', IOSR Journal of Computer Engineering (IOSR-JCE), 17(2), pp. 22–26.
- Kass, D. and Vats, E. (2022) 'AttentionHTR: handwritten text recognition based on attention encoder-decoder networks', in Document Analysis Systems (DAS 2022). Lecture Notes in Computer Science, vol. 13237. Cham: Springer, pp. 507–522. Available at: https://doi.org/10.1007/978-3-031-06555-2_34
- Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z. and Wei, F. (2023) 'TrOCR: transformer-based optical character recognition with pre-trained models', Proceedings of the AAAI Conference on Artificial Intelligence,

37(11), pp. 13094–13102. Available at: https://doi.org/10.1609/aaai.v37i11.26538

- Manchala, S.Y., Kinthali, J., Kotha, K., Kumar, K.S. and Jayalaxmi, J. (2020) 'Handwritten text recognition using deep learning with TensorFlow', International Journal of Engineering Research & Technology (IJERT), 9(5), pp. 585–590.
- Marti, U.-V. and Bunke, H. (2002) 'The IAM-database: an English sentence database for offline handwriting recognition', International Journal on Document Analysis and Recognition, 5(1), pp. 39–46. Available at: https://doi.org/10.1007/s100320200071
- Naveenkumar, M. and Ayyasamy, V. (2016) 'OpenCV for computer vision applications', in Proceedings of the National Conference on Big Data and Cloud Computing (NCBDC). pp. 52–56.
- Parres, D. and Paredes, R. (2023) 'Fine-tuning vision encoder–decoder transformers for handwriting text recognition on historical documents', in Document Analysis and Recognition – ICDAR 2023. Lecture Notes in Computer Science, vol. 14191. Cham: Springer, pp. 253–268. Available at: https://doi.org/10.1007/978-3-031-41685-9_16
- Sahare, P., Dhok, S.B. and Dhok, S. (2018) 'Review of text extraction algorithms for scene-text and document images', IETE Technical Review, 35(3), pp. 1–22.
- Skala, V., Singh, T.P., Choudhury, T., Tomar, R. and Bashar, A. (eds.) (2022) Machine Intelligence and Data Science Applications. Lecture Notes on Data Engineering and Communications Technologies. Singapore: Springer.
- Smith, R. (2007) 'An overview of the Tesseract OCR engine', in Ninth International Conference on Document Analysis and Recognition (ICDAR). IEEE, pp. 629–633. Available at: https://doi.org/10.1109/ICDAR.2007.4376991

Appendices

Appendices

A.1 Model loading — src/htr/models.py

Loads TrOCRProcessor + VisionEncoderDecoderModel from a Hub ID

src > htr > models.py

```python
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Any, Union

import torch
from PIL import Image
from transformers import TrOCRProcessor, VisionEncoderDecoderModel

DEFAULT_DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

TROCR_HANDWRITTEN = "microsoft/trocr-base-handwritten"
LOCAL_TROCR_FINETUNED = "models/iam_trocr_finetuned"

KNOWN_HUB_CHECKPOINTS = {TROCR_HANDWRITTEN}

ImageInput = Union[Image.Image, list[Image.Image]]


@dataclass
class ModelBundle:
    checkpoint: str
    model: VisionEncoderDecoderModel
    device: torch.device
    image_processor: Any
    text_processor: TrOCRProcessor

    def preprocess(self, images: ImageInput) -> torch.Tensor:
        pixel_values = self.image_processor(images, return_tensors="pt").pixel_values
        return pixel_values.to(self.device)

    def generate_batch(self, images: list[Image.Image]) -> list[str]:
        self.model.eval()
        pixel_values = self.preprocess(images)
        with torch.no_grad():
            generated_ids = self.model.generate(pixel_values, max_new_tokens=128)
        return self.text_processor.batch_decode(generated_ids, skip_special_tokens=True)

    def generate(self, image: Image.Image) -> str:
        return self.generate_batch([image])[0]


def _resolve_checkpoint(checkpoint: str) -> tuple[str, Path | None]:
    """Return (load_id, local_path_or_none). Accepts Hub IDs or local dirs."""
    path = Path(checkpoint)
    if path.exists() and path.is_dir():
        return str(path), path
    # Also accept repo-relative models/... even if cwd differs
    repo_models = Path(__file__).resolve().parents[2] / checkpoint
    if repo_models.exists() and repo_models.is_dir():
        return str(repo_models), repo_models
    if checkpoint in KNOWN_HUB_CHECKPOINTS:
        return checkpoint, None
    # Allow any Hub-style id (org/name) for flexibility
    if "/" in checkpoint and not checkpoint.startswith(".") and not checkpoint.startswith("mode
        return checkpoint, None
    raise ValueError(
        f"Unknown checkpoint {checkpoint!r}. "
        f"Use a Hub id (e.g. {TROCR_HANDWRITTEN!r}) or a local directory "
        f"(e.g. {LOCAL_TROCR_FINETUNED!r})."
    )
```

```python
def load_model_bundle(
    checkpoint: str,
    device: torch.device | None = None,
) -> ModelBundle:
    """Load TrOCR checkpoint (Hub or local) with matched processor and weights."""
    load_id, local_path = _resolve_checkpoint(checkpoint)
    resolved_device = device or DEFAULT_DEVICE

    processor_source = load_id
    if local_path is not None:
        # Prefer processor saved with the checkpoint; fall back to Hub base
        has_processor = (local_path / "preprocessor_config.json").exists() or (
            local_path / "tokenizer_config.json"
        ).exists()
        if not has_processor:
            processor_source = TROCR_HANDWRITTEN

    processor = TrOCRProcessor.from_pretrained(processor_source)
    model = VisionEncoderDecoderModel.from_pretrained(load_id)
    model.to(resolved_device)
    model.eval()

    return ModelBundle(
        checkpoint=str(checkpoint),
        model=model,
        device=resolved_device,
        image_processor=processor.image_processor,
        text_processor=processor,
    )
```

A.2 Dataset — src/htr/dataset.py

LineDataset reads split_dir/images/ and labels.csv (filename, transcription). Shared by training and evaluation. collect_charset builds the CRNN vocabulary.

```python
def __init__(
    self,
    split_dir: Path | str,
    max_samples: int | None = None,
    transform: Callable[[Image.Image], Image.Image] | None = None,
    return_pil: bool = False,
):
    self.split_dir = Path(split_dir)
    self.images_dir = self.split_dir / "images"
    self.transform = transform
    self.return_pil = return_pil
    labels_path = self.split_dir / "labels.csv"
    if not labels_path.exists():
        raise FileNotFoundError(f"Missing labels file: {labels_path}")

    self.samples: list[tuple[str, str]] = []
    with open(labels_path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            filename = row["filename"]
            text = row["transcription"]
            image_path = self.images_dir / filename
            if image_path.exists():
                self.samples.append((str(image_path), text))

    if max_samples is not None:
        self.samples = self.samples[:max_samples]
```

A.3 Augmentation — src/htr/augment.py

Train-only torchvision transforms (mild affine + color jitter). Evaluation uses unaugmented RGB images.

```python
from __future__ import annotations

from PIL import Image
from torchvision import transforms


def train_augment() -> transforms.Compose:
    """Mild geometry/brightness transforms for fine-tuning only."""
    return transforms.Compose(
        [
            transforms.RandomAffine(
                degrees=2,
                translate=(0.02, 0.02),
                scale=(0.95, 1.05),
                fill=255,
            ),
            transforms.ColorJitter(brightness=0.15, contrast=0.15),
        ]
    )


def identity_augment(image: Image.Image) -> Image.Image:
    return image
```

A.4 TrOCR fine-tune — src/htr/train.py

AdamW + cross-entropy fine-tuning with early stopping on validation CER.

```python
def _save_bundle(model, processor, output_dir: Path) -> None:
    """Save model+processor with Windows/OneDrive-safe replace (no directory rename)."""
    output_dir = Path(output_dir)
    staging = output_dir.parent / f"{output_dir.name}_staging"
    if staging.exists():
        shutil.rmtree(staging, ignore_errors=True)
        if staging.exists():
            shutil.rmtree(staging)
    staging.mkdir(parents=True, exist_ok=True)
    model.save_pretrained(staging)
    processor.save_pretrained(staging)
    # Copy into place instead of rename: WinError 5 / OneDrive often blocks dir renames.
    if output_dir.exists():
        shutil.rmtree(output_dir, ignore_errors=True)
        if output_dir.exists():
            # Destination locked — write into a sibling then leave staging as fallback.
            alt = output_dir.parent / f"{output_dir.name}_new"
            if alt.exists():
                shutil.rmtree(alt, ignore_errors=True)
            staging.rename(alt)
            raise PermissionError(
                f"Could not replace locked {output_dir}; wrote {alt} instead"
            )
    output_dir.mkdir(parents=True, exist_ok=True)
    for item in staging.iterdir():
        dest = output_dir / item.name
        if item.is_dir():
            if dest.exists():
                shutil.rmtree(dest)
            shutil.copytree(item, dest)
        else:
            shutil.copy2(item, dest)
    shutil.rmtree(staging, ignore_errors=True)
```

A.5 Evaluation — src/htr/evaluate.py

Backends: trocr | tesseract | crnn. Writes metrics.json, predictions.csv, error_summary.

```python
def run_evaluation(config: EvalConfig) -> dict[str, Any]:
    torch.manual_seed(config.seed)
    output_dir = config.output_dir / config.run_id
    output_dir.mkdir(parents=True, exist_ok=True)

    if config.backend == "tesseract":
        import shutil

        if shutil.which("tesseract") is None:
            metrics = {
                "run_id": config.run_id,
                "status": "skipped",
                "reason": "Tesseract binary not installed (sudo apt install tesseract-ocr)",
                "backend": "tesseract",
            }
            with open(output_dir / "metrics.json", "w", encoding="utf-8") as f:
                json.dump(metrics, f, indent=2)
            (output_dir / "README.md").write_text(
                "# Skipped\n\nInstall Tesseract: `sudo apt install tesseract-ocr`\n",
                encoding="utf-8",
            )
            return metrics

    dataset = LineDataset(config.split_dir, config.max_samples)
    if len(dataset) == 0:
        raise ValueError(f"No samples found in {config.split_dir}")

    bundle = None
    crnn_bundle = None
    if config.backend == "trocr":
        bundle = load_model_bundle(config.checkpoint)
    elif config.backend == "crnn":
        from .baselines.crnn import load_crnn_bundle

        crnn_bundle = load_crnn_bundle(config.checkpoint)
```

```python
    predictions: list[str] = []
    references: list[str] = []
    rows: list[dict[str, Any]] = []

    dataloader = DataLoader(
        dataset,
        batch_size=config.batch_size,
        shuffle=False,
        collate_fn=_collate,
    )

    for batch_idx, batch in enumerate(dataloader):
        paths = [item["path"] for item in batch]
        refs = [item["reference"] for item in batch]

        if config.backend == "trocr":
            assert bundle is not None
            preds = _predict_trocr_batch(bundle, paths)
        elif config.backend == "tesseract":
            preds = [_predict_tesseract(p) for p in paths]
        elif config.backend == "crnn":
            assert crnn_bundle is not None
            preds = crnn_bundle.predict_paths(paths)
        else:
            raise ValueError(f"Unknown backend: {config.backend}")

        for path, ref, pred in zip(paths, refs, preds):
            predictions.append(pred)
            references.append(ref)
            rows.append(
                {
                    "filename": Path(path).name,
                    "reference": ref,
                    "prediction": pred,
                    "cer": per_sample_cer(pred, ref),
                }
            )
```

```
        if (batch_idx + 1) % 10 == 0:
            print(f"  [{config.run_id}] {len(predictions)}/{len(dataset)} lines", flush=True)

    metrics = {
        "run_id": config.run_id,
        "checkpoint": config.checkpoint,
        "backend": config.backend,
        "n_samples": len(predictions),
        "cer": compute_cer(predictions, references),
        "wer": compute_wer(predictions, references),
        "split_dir": str(config.split_dir),
    }

    with open(output_dir / "config.json", "w", encoding="utf-8") as f:
        json.dump(asdict(config), f, indent=2, default=str)

    with open(output_dir / "metrics.json", "w", encoding="utf-8") as f:
        json.dump(metrics, f, indent=2)

    with open(output_dir / "predictions.csv", "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["filename", "reference", "prediction", "cer"])
        writer.writeheader()
        writer.writerows(rows)

    error_summary = summarize_errors(predictions, references)
    with open(output_dir / "error_summary.json", "w", encoding="utf-8") as f:
        json.dump(error_summary, f, indent=2)

    readme = (
        f"# {config.run_id}\n\n"
        f"Reproduce:\n\n"
        f"```bash\n"
        f"python scripts/run_eval.py --config configs/{config.run_id}.yaml\n"
        f"```\n"
    )
    (output_dir / "README.md").write_text(readme, encoding="utf-8")

    return metrics
```

A.6 CRNN+CTC — src/htr/baselines/crnn.py

CNN + BiLSTM + CTC baseline.

```python
class CRNN(nn.Module):
    """CNN + BiLSTM + linear head for CTC."""

    def __init__(self, n_classes: int, img_height: int = 32, img_width: int = 128):
        super().__init__()
        self.img_height = img_height
        self.img_width = img_width
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 64, 3, 1, 1),
            nn.ReLU(True),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(64, 128, 3, 1, 1),
            nn.ReLU(True),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(128, 256, 3, 1, 1),
            nn.BatchNorm2d(256),
            nn.ReLU(True),
            nn.Conv2d(256, 256, 3, 1, 1),
            nn.ReLU(True),
            nn.MaxPool2d((2, 1), (2, 1)),
            nn.Conv2d(256, 512, 3, 1, 1),
            nn.BatchNorm2d(512),
            nn.ReLU(True),
            nn.Conv2d(512, 512, 3, 1, 1),
            nn.ReLU(True),
            nn.MaxPool2d((2, 1), (2, 1)),
            nn.Conv2d(512, 512, kernel_size=(2, 2), stride=1, padding=0),
            nn.BatchNorm2d(512),
            nn.ReLU(True),
        )
        feat_dim = _cnn_feat_dim(self.cnn, img_height, img_width)
        self.rnn = nn.LSTM(feat_dim, 256, num_layers=2, bidirectional=True, batch_first=True)
        self.fc = nn.Linear(512, n_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        feats = self.cnn(x)
        b, c, h, w = feats.size()
        feats = feats.permute(0, 3, 1, 2).contiguous().view(b, w, c * h)
        out, _ = self.rnn(feats)
        return self.fc(out)
```

A.7 Metrics — src/htr/metrics.py

Case-sensitive whitespace normalization; corpus CER/WER via jiwer.

```python
import jiwer


def normalize_text(text: str) -> str:
    """Case-sensitive normalization: collapse internal whitespace, strip ends."""
    return re.sub(r"\s+", " ", text.strip())


def compute_cer(predictions: list[str], references: list[str]) -> float:
    preds = [normalize_text(p) for p in predictions]
    refs = [normalize_text(r) for r in references]
    return float(jiwer.cer(refs, preds))


def compute_wer(predictions: list[str], references: list[str]) -> float:
    preds = [normalize_text(p) for p in predictions]
    refs = [normalize_text(r) for r in references]
    return float(jiwer.wer(refs, preds))


def per_sample_cer(prediction: str, reference: str) -> float:
    return compute_cer([prediction], [reference])


def summarize_errors(predictions: list[str], references: list[str], top_n: int = 20) -> dict:
    """Summarize character-level substitution patterns using jiwer alignment."""
    counter: Counter[str] = Counter()

    for pred, ref in zip(predictions, references):
        pred_n = normalize_text(pred)
        ref_n = normalize_text(ref)
        alignment = jiwer.process_words(ref_n, pred_n)
        for chunk in alignment.alignments[0]:
            if chunk.type == "equal":
                continue
            ref_slice = ref_n[chunk.ref_start_idx : chunk.ref_end_idx]
            pred_slice = pred_n[chunk.hyp_start_idx : chunk.hyp_end_idx]
            counter[f"{chunk.type}:{ref_slice!r}->{pred_slice!r}"] += 1

    return {
        "top_errors": [{"pattern": k, "count": v} for k, v in counter.most_common(top_n)],
        "total_mismatched_lines": sum(1 for p, r in zip(predictions, references) if p != r),
    }
```